

Génie logiciel

Livrable 3

CESI 
ÉCOLE D'INGÉNIEURS



ProSoft

Génie Logiciel

Réalisé par

Maéva CHALLIES

Allan BOUHAMED

Enzo CADIÈRE

Maxime STOFFEL

Mathéo CARVAL

Philippe LUU

Projet encadré par

Ali SKAF

Année universitaire 2025-2026

Livrable 3 - 23/02/2026

Informations générales

Versions du document :

Ce tableau retrace toutes les versions du document :

Version	Date	Auteur(s)	Objet
1.0	16/02/2026	LUU Philippe, CARVAL Mathéo, CHALLIES Maéva, CADIÈRE Enzo, STOFFEL Maxime, BOUHAMED Allan	Création du document
1.1	26/02/2026	CARVAL Mathéo, CHALLIES Maéva, CADIÈRE Enzo	Revue du document et des parties : 3 et 4

Liste de diffusion :

Ce tableau répertorie tous les destinataires ayant accès au document :

Entreprise / Organisme	Destinataire(s)	Contacts	Rôle
ProSoft	LUU Philippe	philippe.luu@prosoft.fr	Ingénieur Chef de projet & Scrum Master
ProSoft	CARVAL Matheo	matheo.carval@prosoft.fr	Ingénieur Architecte développement (Lead Dev)
ProSoft	CADIÈRE Enzo	enzo.cadiere@prosoft.fr	Ingénieur DevOps & CI
ProSoft	CHALLIES Maéva	maeva.challies@prosoft.fr	Ingénieure Développement C# & Product Owner (Co-Lead Dev)
ProSoft	BOUHAMED Allan	allan.bouhamed@prosoft.fr	Ingénieur d'Études et Développement C#
ProSoft	STOFFEL Maxime	maxime.stoffel@prosoft.fr	Ingénieur d'Études et Développement C#
CESI	AOUAM Djamel	daouam@cesi.fr	Directeur informatique
CESI	SKAF Ali	askaf@cesi.fr	Responsable informatique

Synthèse Managériale

Ce document présente le bilan technique et fonctionnel de la version 3.0 du logiciel de sauvegarde EasySave, développé par l'équipe d'ingénierie logicielle pour le compte de ProSoft.

L'objectif principal de ce livrable est d'introduire une exécution parallèle des sauvegardes tout en intégrant des mécanismes avancés de contrôle et de régulation, notamment la gestion des fichiers prioritaires, la limitation des transferts volumineux, la pause automatique en cas de détection d'un logiciel métier et la centralisation des logs via Docker. Cette version vise à améliorer les performances globales du système tout en garantissant la stabilité et la cohérence du comportement applicatif.

Les points clés de ce livrable sont l'intégration maîtrisée du parallélisme et le respect strict du cahier des charges. Le logiciel assure désormais l'exécution asynchrone des travaux, le contrôle en temps réel via les commandes Play, Pause et Stop, la gestion du chiffrement via CryptoSoft en mono-instance, ainsi que la centralisation optionnelle des fichiers log. L'ensemble des fonctionnalités demandées pour la version 3.0 est implémenté conformément aux exigences définies.

La maîtrise des coûts, le projet a été livré dans les délais impartis. L'analyse financière estime le coût de R&D à environ 49 000 €, avec un retour sur investissement (ROI) atteignable dès la 244ème licence vendue, confirmant la viabilité économique du produit.

Également, la qualité & maintenance, le respect strict des normes de codage (DRY, Anglais) et la documentation technique fournie assurent une maintenabilité optimale et une reprise aisée par n'importe quelle équipe de ProSoft.

En conclusion, EasySave version 3.0 constitue un socle technologique stable et validé, prêt pour le déploiement des fonctionnalités avancées des prochains jalons.

Sommaire

Informations générales.....	3
Versions du document :	3
Liste de diffusion :	3
Synthèse Managériale.....	4
Sommaire.....	5
1. Contexte et présentation.....	7
1.1 Présentation de ProSoft et du projet EasySave.....	7
1.2 Objectifs & contraintes (cahier des charges).....	7
1.3 Politique tarifaire et maintenance.....	9
2. Gestion de projet et environnement.....	10
2.1 Gestion Git et versioning.....	10
2.2 Workflow Git.....	10
2.3 Suivi du projet.....	11
2.4 Maîtrise d'oeuvre.....	13
2.5 CI/CD.....	14
3. Architecture logistique & choix techniques.....	15
3.1 Architecture MVVM.....	15
3.2 Architecture multithread et parallélisme.....	15
3.3 Architecture EasyLog DLL.....	16
3.4 Architecture du service Docker de centralisation des logs.....	16
3.5 Design patterns utilisés.....	17
4. Spécifications fonctionnelles (V3).....	18
4.1 Sauvegarde parallèle.....	18
4.2 Gestion des fichiers prioritaires.....	18
4.3 Gestion fichiers volumineux.....	18
4.4 Interaction Play / Pause / Stop.....	19
4.5 Détection logiciel métier.....	19
4.6 Gestion CryptoSoft mono-instance.....	20
4.7 Centralisation des logs via Docker.....	20
5. Diagrammes UML.....	22
5.1 Diagramme de classe.....	22
5.2 Diagramme de cas.....	23
5.3 Diagramme de séquence.....	23
5.4 Diagramme d'activité.....	24
6. Ouverture sur la prochaine version.....	26
7. Table des figures.....	28
8. Glossaire.....	29
9. Annexes.....	34
9.1 Guide d'installation et Support Technique (Détails ProSoft).....	34

9.2 Lien vers le dépôt GitHub.....	34
------------------------------------	----

1. Contexte et présentation

1.1 Présentation de ProSoft et du projet EasySave

ProSoft est un éditeur de logiciels reconnu, spécialisé dans le développement de solutions informatiques professionnelles. Dans le cadre de l'élargissement de son portefeuille de produits, l'entreprise lance le projet "EasySave", une solution logicielle de sauvegarde destinée à une clientèle B2B (Business to Business).

ProSoft a pour ambition de fournir un outil fiable, performant et simple d'utilisation pour sécuriser les données critiques des entreprises. Le logiciel EasySave doit permettre la gestion de sauvegardes complètes et différentielles, tout en assurant une traçabilité rigoureuse des opérations via des journaux d'événements. Ce projet s'inscrit dans une démarche évolutive, commençant par une version console (V1.0) pour valider le moteur de sauvegarde, avant d'évoluer vers une interface graphique moderne (V2.0) et des fonctionnalités avancées de parallélisme (V3.0).

1.2 Objectifs & contraintes (cahier des charges)

Le mandat confié à l'équipe de développement porte désormais sur la réalisation de la version 3.0 du logiciel EasySave, une application graphique développée en C# avec le framework .NET 8.0 et reposant sur une architecture MVVM. Cette nouvelle version conserve les fonctionnalités fondamentales des versions précédentes tout en introduisant des capacités avancées de gestion parallèle des sauvegardes, d'optimisation des performances et de contrôle en temps réel des travaux.

Le logiciel doit permettre la configuration et l'exécution d'un nombre illimité de travaux de sauvegarde. Chaque travail est défini par un nom unique, un répertoire source et un répertoire cible, pouvant être situés sur des disques locaux, des périphériques externes ou des lecteurs réseaux. Le système doit continuer à gérer les deux stratégies de sauvegarde, complète et différentielle, en assurant un traitement récursif de l'ensemble des fichiers et sous-répertoires du répertoire source. Contrairement aux versions précédentes, les travaux doivent désormais être exécutés en parallèle, ce qui impose une gestion rigoureuse des accès concurrents aux ressources système et aux fichiers.

Une contrainte fonctionnelle impose la gestion des fichiers prioritaires. L'utilisateur peut définir une liste d'extensions prioritaires dans les paramètres du logiciel. Tant qu'un fichier correspondant à ces extensions est en attente de sauvegarde sur au moins un travail, les fichiers non prioritaires ne doivent pas être transférés. Le logiciel doit également limiter le transfert simultané de fichiers volumineux afin d'éviter la saturation de la bande passante. Une taille maximale configurable permet de définir un seuil au-delà duquel un seul transfert de fichier volumineux peut être effectué à la fois sur l'ensemble des travaux actifs.

Le logiciel doit permettre à l'utilisateur d'interagir en temps réel avec chaque travail de sauvegarde via les fonctions Play, Pause et Stop. La fonction Pause doit interrompre le travail après la fin du transfert du fichier en cours, la fonction Play doit permettre de démarrer ou de reprendre un travail, et la fonction Stop doit interrompre immédiatement l'exécution du travail. Le logiciel doit également surveiller la présence d'un logiciel métier défini dans la configuration. Si ce logiciel est détecté en cours d'exécution, tous les travaux de sauvegarde doivent être automatiquement mis en pause et reprendre uniquement lorsque le logiciel métier est fermé.

Le logiciel doit intégrer le chiffrement des fichiers via l'application externe CryptoSoft, qui est désormais mono-instance. Cela signifie qu'une seule opération de chiffrement peut être exécutée à la fois sur le système. Le logiciel doit donc gérer cette contrainte afin d'éviter tout accès concurrent à CryptoSoft, tout en assurant la continuité des sauvegardes. Seuls les fichiers correspondant aux extensions définies dans les paramètres doivent être chiffrés, et le temps de chiffrement doit être enregistré dans le fichier log journalier.

La fonctionnalité de journalisation repose toujours sur la bibliothèque externe EasyLog.dll, conçue pour rester compatible avec les versions précédentes. Cette bibliothèque doit enregistrer en temps réel les opérations effectuées dans un fichier log journalier au format JSON ou XML selon la configuration utilisateur. En complément, la version 3.0 introduit la possibilité de centraliser les logs sur un serveur Docker. Le logiciel doit pouvoir envoyer les informations de journalisation vers ce serveur afin de permettre une centralisation des logs provenant de plusieurs machines. L'utilisateur peut choisir entre un stockage local, un stockage centralisé ou un stockage hybride combinant les deux modes.

Parallèlement au fichier log, le logiciel doit maintenir un fichier d'état unique permettant de suivre en temps réel la progression des travaux. Ce fichier doit contenir les informations relatives à chaque travail, notamment son état, sa progression, le nombre de fichiers

restants et le fichier en cours de traitement. Ces informations doivent être mises à jour en temps réel afin de permettre une visualisation précise de l'avancement des sauvegardes dans l'interface graphique.

Enfin, des contraintes strictes de maintenabilité, de compatibilité et de qualité logicielle doivent être respectées. Le code doit rester modulaire, maintenable et compatible avec les versions précédentes. L'ensemble des fichiers de configuration, d'état et de journalisation doivent être stockés dans des emplacements persistants compatibles avec les environnements clients et serveurs. L'architecture doit garantir une gestion fiable du parallélisme, des accès concurrents et des ressources système, tout en assurant la stabilité et la performance du logiciel.

1.3 Politique tarifaire et maintenance

EasySave n'est pas seulement un défi technique, c'est un produit commercial soumis à une logique de rentabilité précise. La stratégie de tarification fixée par ProSoft établit un prix unitaire de 200 € HT par licence. À cela s'ajoute un contrat de maintenance facturé annuellement à hauteur de 12 % du prix d'achat, incluant les mises à jour mineures et majeures. Le support technique étant assuré sur une plage horaire standard de 8h à 17h, cinq jours sur sept, il est essentiel de fournir une documentation technique et utilisateur irréprochable, ainsi que des fichiers de logs précis. Cette rigueur a pour objectif de réduire le temps de traitement des tickets support et d'augmenter la marge sur le contrat de support.

2. Gestion de projet et environnement

2.1 Gestion Git et versioning

Le projet EasySave utilise Git avec GitHub comme plateforme d'hébergement et de collaboration, assurant le suivi des modifications, la gestion des versions et le travail en équipe. Le dépôt est privé et les autorisations suivent le principe du moindre privilège, les développeurs créant des branches et soumettant des pull requests, tandis que les responsables valident et fusionnent.

La stratégie repose sur Git Flow, la branche main contient les versions stables destinées à la production, la branche develop regroupe les fonctionnalités en cours avec des versions de test, et des branches dédiées servent aux fonctionnalités, corrections, refactorisations et documentation. Les commits respectent la norme Conventional Commits et le versioning sémantique (MAJOR.MINOR.PATCH) est automatisé par le pipeline CI/CD selon les changements détectés.

Le workflow prévoit la création de branches depuis develop, des commits atomiques, une revue de code avec tests automatiques, puis la fusion par les responsables. Les branches principales sont protégées, les conflits sont résolus manuellement et chaque version génère automatiquement une release avec changelog et exécutables pour plusieurs systèmes, garantissant la traçabilité et la reproductibilité du projet.

2.2 Workflow Git

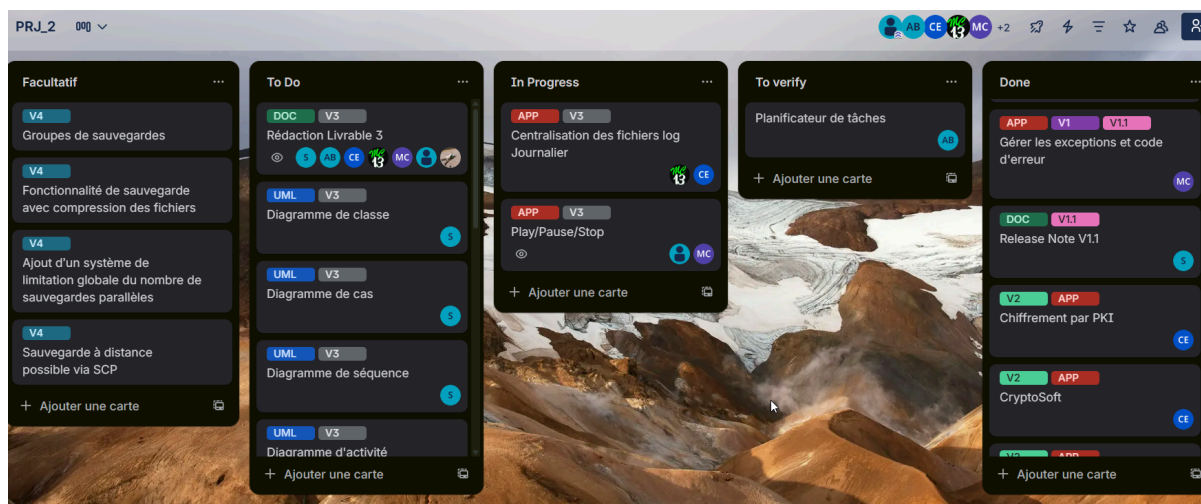
Le processus Git du projet EasySave combine Git Flow avec des adaptations propres au projet afin de maintenir un historique clair, faciliter la collaboration de l'équipe et garantir la qualité du code par des contrôles automatisés. Le cycle de développement commence par la création d'un ticket sur GitHub décrivant le besoin, puis le développeur crée une branche depuis develop, travaille localement avec des commits atomiques conformes aux Conventional Commits et pousse régulièrement ses modifications. Une fois la fonctionnalité terminée, une pull request vers develop est soumise avec description des changements et tests réalisés.

L'ouverture de la pull request déclenche automatiquement le pipeline CI/CD, qui vérifie la compilation, exécute les tests, contrôle le formatage du code et analyse les dépendances. En parallèle, l'équipe effectue une revue du code portant sur la lisibilité, la maintenabilité, la cohérence architecturale et la présence de tests. Après validation, un responsable technique fusionne les modifications par squash and merge, ce qui maintient un historique linéaire.

Les développeurs mettent régulièrement à jour leur branche avec develop afin de limiter les conflits et les résoudre localement. Les versions stables sont publiées par fusion de develop vers main après vérification complète des fonctionnalités, de la documentation et des tests, puis le CI/CD génère automatiquement la version, les artefacts et le journal des modifications. En cas de bug critique en production, un correctif est créé directement depuis main, validé en priorité, puis reporté sur develop afin de conserver la cohérence entre les branches.

2.3 Suivi du projet

Ce troisième livrable marque l'aboutissement du développement de la version 3.0 d'EasySave, intégrant des concepts avancés de programmation système tels que la parallélisation, la gestion des accès concurrentiels et la conteneurisation. Pour mener à bien ce sprint final, nous avons maintenu notre organisation itérative basée sur les boucles PBL, en répartissant les tâches selon les affinités techniques de chaque membre de l'équipe afin d'optimiser notre vélocité. Le suivi quotidien de l'avancement a été rigoureusement documenté sur notre tableau Kanban, ce qui nous a permis de clôturer l'ensemble des développements avec un taux d'achèvement de 100%.



Livrable 3 - 23/02/2026

strictement mono-instance. Ces efforts conjoints nous permettent aujourd'hui de livrer un code performant et parfaitement conforme aux exigences de l'entreprise.

2.4 Maîtrise d'oeuvre

L'évolution vers la version 3.0, marquée par l'intégration complexe de la parallélisation des tâches, la gestion pointue des accès concurrentiels et la centralisation des logs sous Docker, a requis une expertise technique plus poussée de la part de l'équipe. L'abandon du mode séquentiel au profit du multithreading a nécessité un temps de conception et de débogage supplémentaire, calculé sur la base du volume horaire de l'équipe durant ce dernier sprint.

Phase du projet	Entité	Coût entreprise	Volume ingénieur par	Coût total de la phase
Livrable 1	Ingénieur C# (x6)	18.68 €/heure	350 h	39 228.00 €
Livrable 2	Ingénieur C# (x6)	18.68 €/heure	35 h	3 922.80 €
Livrable 3	Ingénieur C# (x6)	18.68 €/heure	50 h	5 604.00 €
Total	XX	XXX	435 h	48 754.80 €

Figure 3 : Tableau de chiffrage cumulé des livrables 1, 2 et 3.

Le coût total de développement du projet EasySave s'élève désormais à 48 754.80 €. En conservant notre politique tarifaire fixée à 200 € HT l'unité, le nouveau seuil de rentabilité est logiquement repoussé et sera atteint à partir d'environ 244 licences vendues. Bien que ce

troisième et dernier sprint ajoute un coût de production au budget global, il transforme fondamentalement le positionnement du logiciel sur le marché.

En effet, l'ajout du support Docker pour la centralisation des journaux et la parallélisation des transferts pour les fichiers non volumineux font d'EasySave une véritable solution d'entreprise (BtoB), capable d'être déployée et administrée sur des parcs informatiques complexes. Cette montée en gamme technique justifie pleinement notre prix de vente de 200 € HT face à la concurrence. De plus, ces fonctionnalités orientées réseau et multi-serveurs constituent un argument de vente majeur pour inciter les clients "grands comptes" à souscrire massivement au contrat de maintenance annuel de 12%, garantissant ainsi des revenus récurrents et une rentabilité pérenne pour ProSoft à moyen et long terme.

2.5 CI/CD

La pipeline que nous avons implémentée pour EasySave permet de publier automatiquement une version spécifique du projet depuis GitHub Actions. Elle commence par valider le format de version (X.Y.Z) et déterminer si la release est une pré-release, puis résout le nom du tag Git correspondant (vX.Y.Z ou vX.Y.Z-rc.1) tout en s'assurant qu'il n'existe pas déjà. Ensuite, elle effectue un checkout complet du code, installe la version .NET 8.0 requise, restaure les packages NuGet en utilisant un cache pour accélérer les builds, compile la solution en configuration Release et exécute les tests unitaires pour garantir que la build est stable avant publication.

Une fois validée, la pipeline crée et pousse le tag Git, puis utilise une stratégie matrix pour générer des builds multiplateformes : Windows (win-x64), Linux (linux-x64) et macOS (osx-arm64). Pour chaque plateforme, elle publie le projet en build self-contained, crée des archives portables (ZIP pour Windows, TAR.GZ pour Linux et macOS) et des installateurs natifs (.msi pour Windows, .dmg pour macOS avec signature ad-hoc, .deb et .rpm pour Linux). Tous les artefacts sont uploadés sur GitHub Actions, puis une release GitHub est générée automatiquement, incluant des notes de version basées sur les commits depuis la dernière release ainsi que tous les fichiers produits, fournissant un package complet prêt à la distribution.

3. Architecture logistique & choix techniques

3.1 Architecture MVVM

La version V3 conserve l'architecture MVVM de la V2, mais elle ajoute un pilotage plus fin de l'exécution des sauvegardes. La séparation est claire entre la vue, qui reste dans Avalonia, le ViewModel qui porte l'état de l'interface, et les services métier. La classe MainViewModel orchestre les commandes utilisateur et s'abonne aux événements du moteur de sauvegarde. Cette organisation se voit dans MainViewModel.cs, où les commandes de lancement, de pause et de reprise sont branchées au gestionnaire métier.

Le binding de progression est renforcé par ProgressViewModel et JobProgressItem, qui maintiennent une progression par tâche et une progression globale pondérée par la taille des données. Le binding est alimenté en continu par les événements FileTransferred émis depuis la couche service, puis relayés sur le thread UI. Cette logique est portée dans ProgressViewModel.cs et MainViewModel.cs. Le binding pause/play est aussi présent avec PauseJobCommand et ResumeJobCommand.

3.2 Architecture multithread et parallélisme

L'architecture multithread repose sur une exécution parallèle par tâche de sauvegarde via Task.Run, à la fois pour un ensemble de tâches et pour les lancements individuels. Le point central est BackupManager, qui expose ExecuteJobAsync et ExecuteJobsInParallelAsync. Chaque travail dispose de son propre PauseToken et le système garde un dictionnaire des tokens actifs pour contrôler plusieurs jobs en même temps. Cette logique est portée par BackupManager.cs.

La contrainte de bande passante des fichiers volumineux est traitée par une régulation globale en octets en vol avec TransferThrottle. Ce composant synchronise les accès entre threads et autorise un fichier au-dessus du seuil seulement s'il n'y a pas déjà un autre transfert en cours qui violerait la limite. L'implémentation se fait entre BackupManager et FileTransferService dans BackupManager.cs et FileTransferService.cs. La pause automatique liée aux logiciels métiers est pilotée par un timer périodique et un drapeau atomique pour éviter les déclenchements concurrents.

3.3 Architecture EasyLog DLL

EasyLog est découpé comme une bibliothèque dédiée, indépendante du cœur métier. Le contrat est porté par ILogger, la classe abstraite LoggerBase implémente la logique de buffering, de flush périodique et de sélection de formater, puis les implémentations concrètes DailyJsonLogger et DailyXmlLogger gèrent l'écriture physique et la rotation journalière. Cette structure est visible dans ILogger.cs, LoggerBase.cs, DailyJsonLogger.cs et DailyXmlLogger.cs.

Dans EasySave, FileTransferService injecte un ILogger et produit une entrée BackupLogEntry à chaque fichier transféré. Le choix de format est dynamique via la configuration et peut être mis à jour sans redémarrage grâce à UpdateLogger. La persistance d'état en temps réel est séparée dans StateWriter, ce qui évite de mélanger le journal métier et l'état d'avancement. Cette partie est portée par FileTransferService.cs, StateWriter.cs et Program.cs.

3.4 Architecture du service Docker de centralisation des logs

Dans l'état actuel du dépôt, la centralisation Docker n'est pas implémentée côté code applicatif. Les logs sont écrits localement via EasyLog, avec rotation journalière. L'architecture V3 cible pour la centralisation doit ajouter un composant de publication réseau entre EasySave et un service Docker, avec trois modes de fonctionnement: local seul, centralisé seul, et double écriture local plus centralisé.

Techniquement, le point propre pour l'intégration est entre FileTransferService et ILogger, en ajoutant un logger composite ou un dispatcher de logs. Ce dispatcher envoie les entrées vers un endpoint HTTP du service Docker et gère la tolérance aux pannes réseau avec une file tampon locale. Le service Docker reçoit les événements en temps réel, les persiste, puis expose une API de consultation. Cela permet de rester compatible avec l'architecture existante sans casser la logique métier.

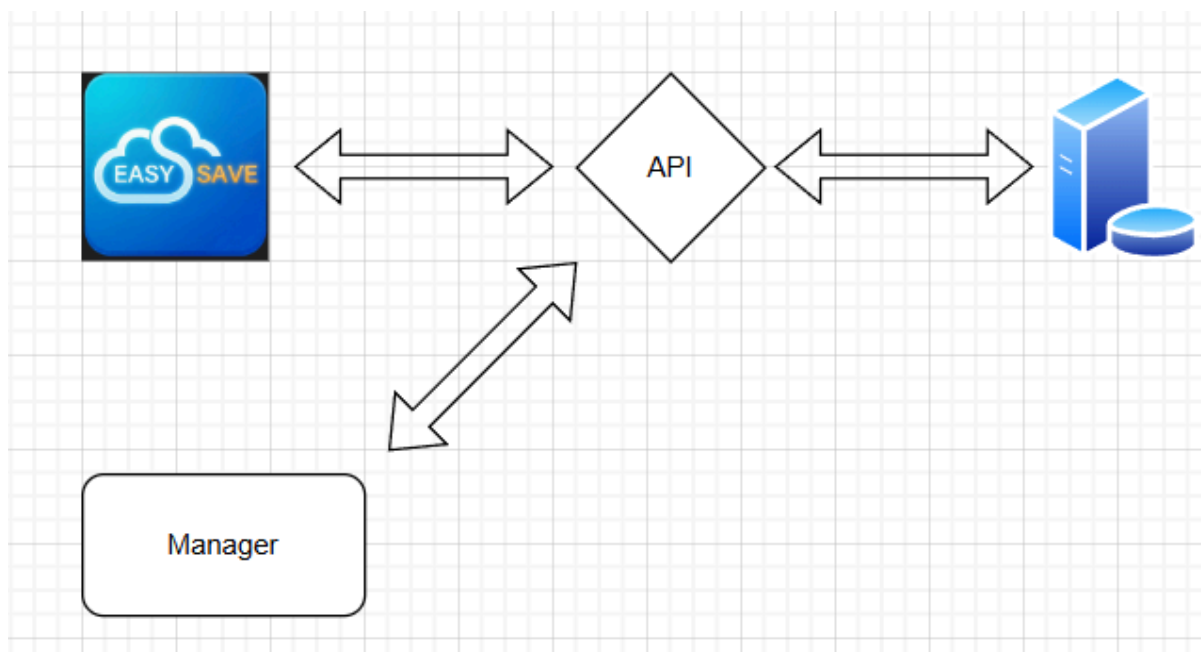


Figure 4 : Schéma du fonctionnement de l'API

3.5 Design patterns utilisés

Le pattern MVVM structure la couche présentation, avec des ViewModels orientés état et des commandes RelayCommand pour déclencher les actions. Le pattern Observer est utilisé via les événements FileTransferred, JobAutoPaused et JobAutoResumed, qui découlent de la production des événements métier et la mise à jour de l'UI. Le pattern Singleton est appliqué à ConfigurationManager pour centraliser la configuration runtime.

On retrouve aussi une logique proche de Strategy dans EasyLog, avec sélection de formatters selon le type et le format, et une Template Method dans LoggerBase avec des méthodes abstraites WriteToFile et ReadFromFile spécialisées par logger. Enfin, la synchronisation concurrente combine des primitives bas niveau, comme ManualResetEventSlim, lock, Monitor et Interlocked, pour assurer une exécution parallèle contrôlée.

4. Spécifications fonctionnelles (V3)

4.1 Sauvegarde parallèle

La V3 exécute les travaux en parallèle et abandonne le mode strictement séquentiel en interface graphique. L'exécution simultanée est visible dans MainViewModel, qui lance plusieurs jobs dans des tâches séparées, et dans BackupManager, qui propose ExecuteJobsInParallelAsync. Cette évolution est cohérente avec l'objectif de réduction du temps global de sauvegarde quand plusieurs travaux sont sélectionnés.

Chaque tâche conserve son propre état de progression et son cycle de vie. Les erreurs sont isolées par job, puis remontées à l'interface avec un marquage spécifique. La conception garde donc un parallélisme effectif sans fusionner les contextes d'erreur ou de progression.

4.2 Gestion des fichiers prioritaires

La gestion des extensions prioritaires est paramétrable dans les réglages, persistée en configuration, puis propagée au moteur de transfert à chaud après sauvegarde des paramètres. Les fichiers sont triés par ordre de priorité avant transfert dans FileTransferService. La configuration est donc bien intégrée de bout en bout entre SettingsViewModel, Configuration et BackupManager.

La limite actuelle est que la priorité est appliquée dans la liste de fichiers d'un job donné. La règle métier la plus stricte, qui interdit tout fichier non prioritaire tant qu'une extension prioritaire reste en attente sur au moins un autre job, n'est pas totalement garantie de façon globale inter-jobs. Cette partie demande un arbitre global de planification pour être parfaitement conforme à la spécification.

4.3 Gestion fichiers volumineux

La gestion des gros fichiers repose sur un seuil configurable en unité KB, MB, GB ou TB, stocké en configuration. Ce seuil est converti en octets puis injecté dans TransferThrottle. Le throttle maintient la quantité d'octets en vol entre jobs parallèles et bloque les acquisitions qui dépassent la capacité autorisée.

Le comportement demandé est respecté dans son principe: pendant un transfert volumineux, d'autres fichiers plus petits peuvent continuer à passer si la contrainte globale de capacité est respectée. Cette implémentation protège la bande passante sans revenir à une exécution totalement séquentielle.

4.4 Interaction Play / Pause / Stop

La lecture fonctionnelle montre que Play et Pause sont opérationnels. Pause ne coupe pas brutalement un fichier en cours, car la suspension est vérifiée entre les opérations de transfert via `WaitIfPaused`. Reprise restaure l'état actif et relance l'avancement normal. La progression est visible en temps réel avec un pourcentage, un fichier courant et des indicateurs complémentaires dans `ProgressViewModel`.

En revanche, Stop immédiat n'est pas finalisé dans la version courante. L'architecture n'injecte pas encore de `CancellationToken` jusqu'au niveau de la boucle de copie. Le besoin fonctionnel V3 est identifié, mais sa fermeture complète nécessite une interruption explicite en cours de transfert et un état métier dédié à l'arrêt.

4.5 Détection logiciel métier

La détection de logiciels métiers est active et s'appuie sur un scan périodique des processus système. Si une application bloquante est détectée, tous les jobs en cours passent en pause automatique. Quand l'application n'est plus détectée, les jobs auto-pausés reprennent automatiquement. Cette logique est sécurisée contre les doubles déclenchements par un drapeau atomique dans `BackupManager`.

La liste des processus bloquants est configurable dans les paramètres. Le nom de processus est normalisé pour éviter les incohérences entre saisie et nom réel. Le comportement observé est conforme à l'exigence de pause temporaire obligatoire tant qu'un logiciel métier est actif.

4.6 Gestion CryptoSoft mono-instance

CryptoSoft est un outil de chiffrement en ligne de commande développé en .NET 8, conçu pour sécuriser des sauvegardes automatisées à l'aide d'un schéma cryptographique hybride. Il combine le chiffrement symétrique AES-256-GCM pour les données et le chiffrement asymétrique RSA-4096 OAEP-SHA256 pour protéger la clé de session : à chaque fichier chiffré, une clé AES éphémère est générée aléatoirement, utilisée pour chiffrer les données par blocs de 64 Kio, puis elle-même chiffrée avec la clé publique RSA du destinataire. Cette approche garantit confidentialité, intégrité (via les tags GCM 128 bits par bloc) et anti-rejeu (nonces uniques par fichier et par bloc), sans jamais recourir à un mot de passe.

Sur le plan pratique, CryptoSoft s'intègre aussi bien en CLI qu'en tant que bibliothèque .NET via une API publique (CryptoSoftApi). Il supporte le chiffrement de fichiers, de dossiers entiers (compressés à la volée) et des flux stdin/stdout pour s'insérer dans des pipelines. Ses performances sont optimisées pour les gros volumes grâce au traitement en streaming, une empreinte mémoire constante d'environ 200 Kio quelle que soit la taille du fichier, et l'exploitation des instructions matérielles AES-NI qui permettent de chiffrer 1 Gio en environ 2 secondes sur un CPU moderne. La gestion des clés suit les bonnes pratiques de sécurité : permissions restrictives sur la clé privée, effacement sécurisé des buffers en mémoire via `CryptographicOperations.ZeroMemory()`, et compatibilité avec des solutions de stockage sécurisé comme les HSM ou Azure Key Vault.

4.7 Centralisation des logs via Docker

Afin de répondre au besoin de centralisation des journaux d'activité, l'architecture retenue repose sur la mise en place d'une API distante dédiée à la collecte des logs. Cette API est déployée au sein d'un conteneur Docker, permettant une installation standardisée, portable et facilement déployable sur différentes infrastructures.

L'API est développée en ASP.NET et expose des endpoints sécurisés permettant à l'application EasySave d'envoyer en temps réel les événements de journalisation. Cette approche permet de centraliser les logs provenant de plusieurs instances du logiciel,

facilitant ainsi la supervision, l'audit et l'analyse des opérations de sauvegarde à l'échelle d'un parc informatique.

L'accès à cette API est sécurisé via un mécanisme d'authentification par token. Un système de gestion des rôles permet de différencier les niveaux d'accès (consultation, administration, supervision), garantissant ainsi un contrôle strict des droits utilisateurs. Cette organisation assure à la fois la sécurité des données de journalisation et la traçabilité des accès.

Cette solution offre une architecture évolutive et compatible avec les environnements professionnels, tout en conservant la possibilité d'un fonctionnement hybride combinant stockage local et centralisé selon la configuration choisie.

5. Diagrammes UML

5.1 Diagramme de classe

Le diagramme de classes du livrable 3 met en évidence l'évolution d'EasySave vers une architecture capable de gérer l'exécution parallèle des sauvegardes tout en intégrant de nouvelles contraintes métier liées aux priorités, aux fichiers volumineux et à la centralisation des logs. L'architecture conserve la séparation entre la couche de présentation, la logique métier et les services techniques, tout en introduisant des mécanismes de synchronisation adaptés au parallélisme.

La classe BackupJob reste l'entité centrale représentant un travail de sauvegarde. Elle contient les informations relatives aux répertoires source et cible, au type de sauvegarde, ainsi qu'à l'état courant du travail et à sa progression. Dans la version 3.0, cette classe est enrichie par des attributs permettant de suivre le statut d'exécution en parallèle et les états intermédiaires tels que Active, Paused ou Stopped.

Le BackupManager joue un rôle central dans l'orchestration des travaux. Il initialise les exécutions asynchrones et coordonne les différents JobRunner responsables du traitement effectif des fichiers. Les règles spécifiques de la version 3.0, notamment la gestion des fichiers prioritaires et la limitation des transferts volumineux, sont intégrées au niveau des services de traitement. Des mécanismes de synchronisation assurent le respect des contraintes globales, comme l'interdiction de transférer simultanément plusieurs fichiers dépassant le seuil défini.

L'intégration de CryptoSoft est représentée comme un service externe utilisé via un gestionnaire dédié. Le système garantit son fonctionnement en mono-instance grâce à un mécanisme de verrouillage, évitant tout accès concurrent.

La bibliothèque EasyLog.dll reste indépendante et continue d'assurer la journalisation. La centralisation via Docker est modélisée comme un service externe complémentaire pouvant recevoir les entrées de log selon le mode configuré. Cette organisation confirme que la version 3.0 renforce les capacités du système sans remettre en cause son architecture modulaire initiale.

5.2 Diagramme de cas

Le diagramme de cas d'utilisation du livrable 3 met en évidence l'évolution fonctionnelle du système vers une exécution parallèle des sauvegardes et une interaction en temps réel avec chaque travail.

L'utilisateur demeure l'unique acteur du système. Il peut gérer les travaux de sauvegarde, les créer, les modifier et les supprimer. Il peut également lancer l'exécution des sauvegardes, qui se font désormais en parallèle, ce qui constitue une évolution majeure par rapport aux versions précédentes.

Le diagramme met en évidence l'introduction des nouvelles règles métier. Lors du lancement d'une sauvegarde, le système applique automatiquement la règle des fichiers prioritaires. Tant que des fichiers prioritaires sont en attente, les fichiers non prioritaires ne peuvent être traités. Le système applique également la limitation liée aux fichiers volumineux afin d'éviter une saturation des ressources.

Le chiffrement via CryptoSoft est intégré au processus d'exécution. Il intervient uniquement pour les extensions définies dans les paramètres et respecte la contrainte de mono-instance. Le système inclut également un mécanisme de pause automatique en cas de détection d'un logiciel métier actif.

Enfin, la centralisation des logs via Docker apparaît comme une extension du processus de journalisation. Le système peut écrire les logs localement, les envoyer vers un serveur centralisé, ou combiner les deux modes selon la configuration choisie.

Ce diagramme montre que la version 3.0 introduit des fonctionnalités avancées tout en conservant un acteur unique et une interaction simple du point de vue utilisateur.

5.3 Diagramme de séquence

Les diagrammes de séquence du livrable 3 décrivent les interactions dynamiques liées à l'exécution parallèle des sauvegardes et à l'application des nouvelles contraintes métier.

Lorsqu'un utilisateur lance plusieurs travaux, l'interface graphique transmet la demande au BackupManager. Celui-ci initialise les tâches asynchrones correspondant à chaque travail sélectionné. Chaque JobRunner traite les fichiers de manière indépendante tout en respectant les contraintes globales.

Pour chaque fichier, le système vérifie d'abord la règle des priorités. Si un fichier prioritaire est présent dans la file globale, les autres fichiers sont temporairement suspendus. Ensuite, la règle concernant les fichiers volumineux est appliquée. Si la taille du fichier dépasse le seuil configuré, un mécanisme de verrouillage empêche l'exécution simultanée d'un autre transfert volumineux.

Lorsque le fichier nécessite un chiffrement, le gestionnaire appelle CryptoSoft. Un verrou global garantit que l'application externe fonctionne en mono-instance. Une fois le chiffrement terminé, le processus de sauvegarde se poursuit normalement.

Les diagrammes de séquence décrivent également le comportement des commandes Play, Pause et Stop. La commande Pause attend la fin du transfert en cours avant de suspendre l'exécution, tandis que la commande Stop interrompt immédiatement le traitement. En cas de détection d'un logiciel métier, le système déclenche automatiquement une pause des travaux actifs.

Ces séquences mettent en évidence la coordination entre les composants et la gestion du parallélisme introduite dans la version 3.0.

5.4 Diagramme d'activité

Les diagrammes d'activité du livrable 3 décrivent le flux global d'exécution des sauvegardes en parallèle ainsi que l'application des nouvelles règles métier.

Lorsqu'un utilisateur lance l'exécution des travaux, le système initialise les tâches asynchrones pour chaque sauvegarde sélectionnée. Chaque travail entre dans une boucle de traitement des fichiers. Avant chaque transfert, le système vérifie la présence éventuelle d'un logiciel métier actif. Si celui-ci est détecté, les travaux sont automatiquement mis en pause jusqu'à la fermeture du logiciel.

Pour chaque fichier, le système applique successivement la règle des priorités et la limitation des fichiers volumineux. Si les conditions sont respectées, le fichier est transféré. Si le chiffrement est requis, l'application CryptoSoft est invoquée, en respectant la contrainte de mono-instance.

À chaque étape, le système met à jour le fichier d'état afin de refléter la progression en temps réel dans l'interface graphique. Les logs sont générés simultanément, soit localement, soit via le service Docker selon la configuration.

Ces diagrammes d'activité montrent que la version 3.0 repose sur une exécution asynchrone structurée, intégrant des contrôles de synchronisation et des règles métier globales. Ils confirment que l'évolution vers le parallélisme a été intégrée sans compromettre la cohérence et la modularité du système

6. Ouverture sur la prochaine version

Plusieurs améliorations peuvent être envisagées pour une future version 4.0 afin de renforcer les capacités du logiciel et répondre à de nouveaux besoins utilisateurs.

Une première évolution pertinente consiste à intégrer un planificateur de tâches. Cette fonctionnalité permettrait à l'utilisateur de programmer automatiquement l'exécution des sauvegardes à des horaires définis, par exemple quotidiennement ou hebdomadairement. Cela permettrait d'automatiser entièrement les sauvegardes sans intervention manuelle, ce qui est particulièrement utile dans un environnement professionnel où les sauvegardes doivent être effectuées en dehors des heures de travail. Le temps de développement de cette fonctionnalité reste modéré, car elle peut s'appuyer sur l'architecture existante et sur des composants standards du framework .NET, tout en apportant un bénéfice important pour l'utilisateur en termes d'automatisation et de sécurité des données.

Une autre amélioration envisagée concerne l'ajout d'un système de limitation globale du nombre de sauvegardes parallèles. Bien que la version 3.0 permette l'exécution parallèle des travaux, il peut être pertinent de limiter dynamiquement le nombre de sauvegardes actives afin d'éviter une surcharge du processeur ou du disque. Cette fonctionnalité permettrait d'adapter automatiquement le nombre de sauvegardes simultanées en fonction des ressources disponibles sur la machine. Cette optimisation améliorerait la stabilité du système et les performances globales, avec un temps de développement raisonnable basé sur les mécanismes de synchronisation déjà présents dans la version 3.0.

Il serait également pertinent d'ajouter une fonctionnalité de sauvegarde avec compression des fichiers. Cette évolution permettrait de réduire l'espace de stockage utilisé sur le répertoire cible et de limiter l'utilisation de la bande passante lors des transferts, notamment pour les sauvegardes sur des disques réseaux. Cette fonctionnalité apporterait un bénéfice direct pour les clients disposant d'un espace de stockage limité. Le temps de développement est modéré, car il est possible d'utiliser les bibliothèques de compression déjà disponibles dans .NET.

Une autre évolution importante serait la possibilité d'effectuer des sauvegardes à distance via le protocole SCP. Cette fonctionnalité permettrait à l'utilisateur de sauvegarder ses fichiers sur un serveur distant sécurisé, accessible via le réseau. Cela permettrait de

répondre aux besoins des entreprises disposant d'infrastructures distribuées ou souhaitant externaliser leurs sauvegardes sur des serveurs dédiés. L'utilisation du protocole SCP garantirait un transfert sécurisé des données grâce au chiffrement intégré. Cette fonctionnalité pourrait être implémentée en s'appuyant sur des bibliothèques existantes compatibles avec .NET, telles que SSH.NET, ce qui permettrait de limiter le temps de développement. Le bénéfice pour l'utilisateur serait important, car cela offrirait une solution de sauvegarde distante fiable et sécurisée, tout en élargissant les cas d'utilisation du logiciel EasySave.

Concernant l'optimisation des sauvegardes parallèles, cette approche présente un bénéfice important en termes de performance, notamment lorsque plusieurs travaux doivent être exécutés. L'exécution parallèle permet d'utiliser plus efficacement les ressources du processeur et de réduire le temps total nécessaire pour effectuer les sauvegardes. Toutefois, cette approche nécessite une gestion rigoureuse des accès concurrents afin d'éviter les conflits ou la surcharge du système. La version 3.0 introduit des mécanismes permettant de contrôler ces accès, notamment pour les fichiers volumineux et le chiffrement via CryptoSoft. L'ajout de mécanismes supplémentaires, comme la limitation dynamique du nombre de sauvegardes simultanées, permettra d'améliorer encore la stabilité et les performances dans les versions futures.

7. Table des figures

Figure 1 : Extrait du trello pour la V3 de EasySave.....	12
Figure 2 : Extrait du diagramme de Gantt du livrable 3 de EasySave.....	12
Figure 3 : Tableau de chiffrage cumulé des livrables 1, 2 et 3.....	13
Figure 4 : Schéma du fonctionnement de l'API.....	17

8. Glossaire

AES-256-GCM : Algorithme de chiffrement symétrique (AES 256 bits) en mode GCM assurant confidentialité + intégrité (tag d'authentification).

API (Application Programming Interface) : Interface permettant à deux logiciels de communiquer via des endpoints, méthodes et formats de données.

Artefact : Élément produit par le projet (exécutable, package, documentation, diagramme, log, etc.).

ASP.NET : Framework .NET pour développer des applications web et des API HTTP.

Asynchrone : Mode d'exécution où une tâche s'effectue sans bloquer le thread appelant (permet de garder l'UI réactive).

Avalonia : Framework UI multiplateforme .NET utilisé pour l'interface graphique d'EasySave.

Backup (Sauvegarde) : Copie de données d'un répertoire source vers un répertoire cible pour assurer restauration et continuité.

Backup différentiel : Sauvegarde ne copiant que les fichiers modifiés depuis la dernière sauvegarde complète (selon la stratégie retenue).

Backup complet : Sauvegarde copiant l'intégralité des fichiers du répertoire source.

B2B (Business to Business) : Modèle commercial où le produit est vendu à des entreprises plutôt qu'à des particuliers.

Bande passante : Débit maximal de transfert de données (réseau ou disque). Peut être saturée par des transferts volumineux.

Binding (Data Binding) : Mécanisme MVVM liant automatiquement des données du ViewModel aux éléments de la vue.

CI/CD : Chaîne automatisée d'intégration continue (tests, build) et de déploiement/livraison continue (packaging, release).

CLI (Command Line Interface) : Interface en ligne de commande (terminal) utilisée pour exécuter CryptoSoft.

Conventional Commits : Convention de messages de commit (ex: `feat:`, `fix:`) facilitant changelog et versioning automatiques.

Docker : Technologie de conteneurisation permettant de packager et exécuter un service (ex: centralisation des logs) de façon portable.

DRY (Don't Repeat Yourself) : Principe de conception visant à éviter la duplication de code et centraliser la logique.

EasyLog.dll : Bibliothèque de journalisation utilisée par EasySave pour écrire les logs (JSON/XML) avec rotation journalière.

Endpoint : URL exposée par une API (ex: `/logs`) pour envoyer ou consulter des données.

État (State file) : Fichier unique qui reflète en temps réel l'avancement des travaux (progression, fichier courant, restants, statut).

Extension de fichier : Suffixe d'un fichier (ex: `.pdf`, `.txt`) utilisé pour filtrer priorités et chiffrement.

Fichier prioritaire : Fichier dont l'extension appartient à une liste définie ; il doit être traité avant les fichiers non prioritaires.

Fichier volumineux (gros fichier) : Fichier dépassant un seuil de taille configuré ; son transfert est limité globalement (ex: un seul à la fois).

Git : Système de gestion de versions distribué pour historiser, collaborer et revenir sur des changements.

Git Flow : Stratégie de branches (souvent `main`, `develop`, `feature/*`, `hotfix/*`) pour organiser développement et releases.

GitHub Actions : Service CI/CD de GitHub exécutant workflows (build, tests, packaging, release).

Gantt : Diagramme de planification représentant les tâches dans le temps.

Hybride (stockage hybride) : Mode combinant logs locaux + centralisés (Docker) selon la configuration.

ILogger : Contrat (interface) de journalisation (EasyLog) permettant d'abstraire l'écriture des logs.

JSON (JavaScript Object Notation) : Format texte structuré utilisé pour l'écriture des logs et échanges API.

Kanban : Méthode de suivi visuel des tâches (colonnes type To Do / Doing / Done), souvent via Trello/GitHub Projects.

Lead Dev : Responsable technique/architectural qui oriente les choix techniques et valide des décisions clés.

Lock / Verrou : Mécanisme de synchronisation empêchant plusieurs threads d'accéder simultanément à une ressource critique.

Log (journal) : Trace horodatée des événements (transferts, erreurs, temps de chiffrement) utilisée pour audit et support.

Main / Develop : Branches Git principales : **main** (stable / production), **develop** (intégration des features avant release).

Maintenance logicielle : Activités après livraison (corrections, évolutions, support), ici facturée annuellement.

Mono-instance : Contrainte imposant qu'un seul processus/opération ne puisse s'exécuter à la fois (CryptoSoft).

Monitor / Interlocked / ManualResetEventSlim : Primitives .NET pour gérer synchronisation, atomicité et pause/reprise multi-threads.

MVVM (Model–View–ViewModel) : Architecture UI séparant modèle métier, vue et logique de présentation (liaisons + commandes).

Parallélisme : Exécution simultanée de plusieurs travaux (jobs) via threads/tâches afin de réduire le temps global.

PauseToken : Mécanisme de pause/reprise contrôlant l'exécution d'un job sans tuer le thread (pause après fin du fichier courant).

Pipeline : Chaîne automatisée d'étapes (build → tests → packaging → release) exécutée par la CI/CD.

Play / Pause / Stop : Commandes de contrôle d'un job : démarrer/reprendre, suspendre proprement, arrêter immédiatement.

Pull Request (PR) : Demande de fusion d'une branche vers une autre avec revue de code et contrôles CI.

Récursif : Parcours automatique des sous-dossiers et fichiers d'un répertoire source.

Release : Version publiée (tag + artefacts + notes) générée automatiquement via CI/CD.

ROI (Return On Investment) : Indicateur de rentabilité : moment où les recettes couvrent les coûts (seuil de rentabilité).

RSA-4096 OAEP-SHA256 : Algorithme asymétrique (clé 4096 bits) utilisé pour chiffrer une clé de session (OAEP + SHA-256).

Scrum Master : Rôle facilitant l'organisation agile (rituels, obstacles, coordination), ici côté encadrement projet.

Self-contained : Publication .NET incluant le runtime, permettant d'exécuter l'application sans installer .NET sur la machine cible.

Semaphore / Throttle (TransferThrottle) : Mécanisme de régulation limitant les transferts (ex: gros fichiers) en parallèle.

Singleton : Pattern garantissant une seule instance d'un objet (ex: gestionnaire de configuration).

Squash and merge : Fusion Git combinant plusieurs commits d'une branche en un seul commit dans la branche cible.

Streaming : Traitement des données par flux (blocs) pour limiter la mémoire et gérer de très gros fichiers.

Tag (Git) : Marqueur pointant sur un commit (ex: `v3.0.0`) servant à identifier une version.

Task / Task.Run : Abstraction .NET d'un traitement asynchrone pouvant être planifié sur le threadpool.

Thread : Fil d'exécution ; plusieurs threads permettent l'exécution parallèle mais nécessitent synchronisation.

Token (authentication) : Jeton utilisé pour sécuriser l'accès à une API (ex: en-tête HTTP).

Traçabilité : Capacité à retrouver qui a fait quoi, quand, et comment (via logs, versioning, issues, releases).

UML : Langage de modélisation (diagrammes de classes, cas d'utilisation, séquence, activité) décrivant structure et comportements.

XML (eXtensible Markup Language) : Format texte structuré alternatif à JSON pour la journalisation.

9. Annexes

9.1 Guide d'installation et Support Technique (Détails ProSoft)

cf. Manuel_EasySave_v3.0.pdf

9.2 Lien vers le dépôt GitHub

<https://github.com/MatheoCarval/Projet-EasySave>